1. Project Overview

Problem & Motivation: Manual code reviews are time-consuming, prone to human error, and often miss subtle logical or security flaws. Developers require a tool that accelerates this process while providing deep, context-aware analysis.

Target Users: Software developers, QA engineers, and project leads.

Agent Capabilities: The AI Code Review Agent analyzes source code to generate structured findings, summarize functionality, detect bugs, check coding standards, and suggest refactoring and tests. It also captures human feedback to adapt to project-specific conventions.

Why an AI Agent is Appropriate: Unlike deterministic linters, an AI agent utilizes reasoning, context retrieval, and decision-making to identify complex logical issues, explain potential bugs in natural language, and propose functional multi-step refactoring solutions.

AI/LLM Model: The system will integrate GPT-4o (or a comparable model like Claude 3.5 Sonnet) via API calls.

Architecture Interaction: The GUI and CLI interact with an AgentController. The Controller coordinates the RepoManager (for file operations) and the ReviewAgent (a facade interacting with the LLMClient), resulting in structured review findings returned to the UI.

2. Detailed Feature Specifications

Below are 10 major, non-trivial features integrated into the GUI/CLI workflows.

1. **F01 - Repository Import & File Browsing**
    - **Description:** Imports a local or remote Git repository and maps its structure.
    - **User Interaction:** User inputs a path/URL in the GUI repository browser.
    - **Input:** File path or Git URL.
    - **Output:** Hierarchical file tree display.
    - **AI Involvement:** Deterministic.
    - **Workflow:** System validates the path, clones/reads the directory, and populates the file browser UI.
    - **Error Cases:** Invalid path or permission denied; system prompts user to verify access.
2. **F02 - Code Summarization**
    - **Description:** Generates a high-level natural language summary of selected code or files.

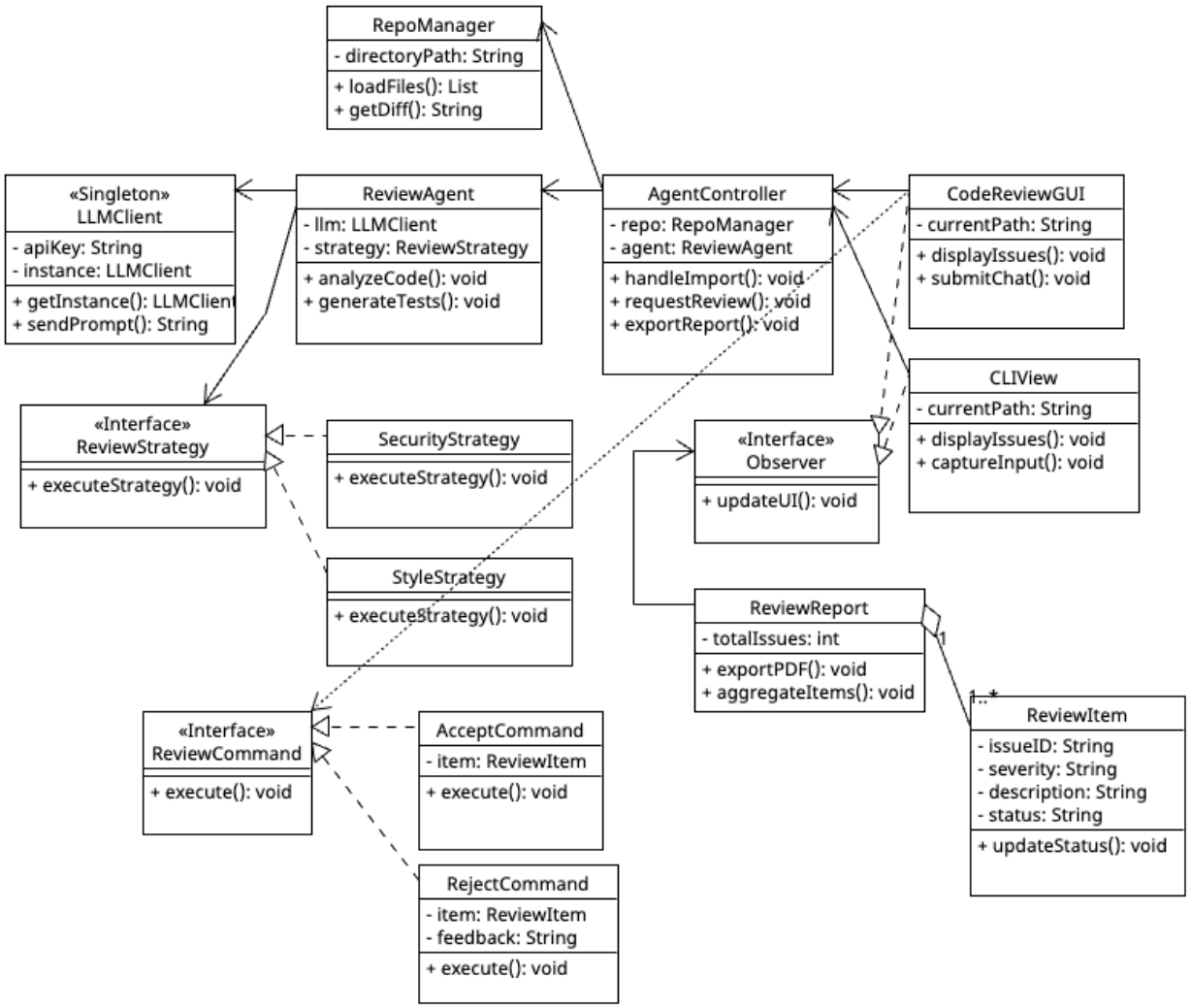
- **User Interaction:** User highlights code in the viewer and clicks "Summarize".
- **Input:** Raw source code strings.
- **Output:** Textual summary of logic and dependencies.
- **AI Involvement:** AI-based.
- **Workflow:** GUI sends code to AgentController → LLMClient → displays result in the review panel.
- **Error Cases:** Code block too large for context window; system prompts to select a smaller snippet.

3. **F03 - Issue Detection & Severity Categorization**
   - **Description:** Scans code to detect potential bugs, categorizing them by severity (e.g., Critical, Warning, Info).
   - **User Interaction:** Triggered manually via "Analyze" button or automatically on file open.
   - **Input:** Source code and language context.
   - **Output:** List of issues with line numbers and severity tags.
   - **AI Involvement:** AI-based.
   - **Workflow:** Agent processes code, queries LLM for vulnerabilities, maps outputs to severity structures, and populates the issue panel.
   - **Error Cases:** API timeout; system retries or alerts user.
4. **F04 - Refactoring Suggestions**
   - **Description:** Proposes optimized, cleaner, or more efficient code replacements.
   - **User Interaction:** User clicks an identified issue and selects "Suggest Refactor".
   - **Input:** Code snippet and the specific detected issue.
   - **Output:** A side-by-side code diff.
   - **AI Involvement:** AI-based.
   - **Workflow:** LLM generates a rewritten snippet. GUI renders a comparison view.
   - **Error Cases:** Malformed code response from LLM; system requests a regeneration.
5. **F05 - Test Generation Suggestion**
   - **Description:** Creates unit test cases for complex or untested functions.
   - **User Interaction:** User clicks "Generate Tests" on a selected function.
   - **Input:** Function signature, body, and existing test framework context.
   - **Output:** Executable unit test code block.
   - **AI Involvement:** AI-based.
   - **Workflow:** Agent identifies edge cases, queries LLM to write tests, and presents them for copying/saving.
   - **Error Cases:** Unrecognized testing framework; system defaults to standard language libraries.
6. **F06 - Coding-Standard Checking**
   - **Description:** Validates code against predefined project style guidelines.
   - **User Interaction:** Configured in settings; runs concurrently with Issue Detection.
   - **Input:** Code files and a .rules configuration file.
   - **Output:** Style violation warnings.
   - **AI Involvement:** Hybrid (Deterministic parsing + AI interpretation of vague rules).

    - **Workflow:** System parses strict rules deterministically and uses AI for naming conventions or stylistic nuances.
    - **Error Cases:** Missing rule file; system uses default PEP8/Google standards.
7. **F07 - Review Acceptance/Rejection & Human Feedback Capture**
    - **Description:** Allows users to accept or dismiss AI suggestions, capturing reasoning for rejected reviews.
    - **User Interaction:** User clicks "Accept" or "Reject" with an optional text reason.
    - **Input:** User button click and text feedback.
    - **Output:** Updated issue status and saved feedback log.
    - **AI Involvement:** Deterministic.
    - **Workflow:** GUI registers the command, updates the database/repository, and clears the UI panel.
    - **Error Cases:** Database write failure; system caches action locally and retries.
8. **F08 - Comparison of Reviews Across Versions**
    - **Description:** Tracks how code quality and AI findings change between commits.
    - **User Interaction:** User selects two commits in the repository browser to compare.
    - **Input:** Commit Hashes A and B.
    - **Output:** Delta report showing resolved and new issues.
    - **AI Involvement:** Hybrid.
    - **Workflow:** System fetches diffs, runs historical data comparison, and uses AI to summarize the quality shift.
    - **Error Cases:** Commits do not exist; system throws a "Git Reference Error".
9. **F09 - Review-History Tracking & Report Generation**
    - **Description:** Compiles all findings, accepted/rejected states, and code summaries into a structured document.
    - **User Interaction:** User clicks "Export Report" and selects a format (PDF/Markdown).
    - **Input:** Current project review session data.
    - **Output:** Downloadable file.
    - **AI Involvement:** Deterministic.
    - **Workflow:** System aggregates the ReviewReport objects and formats them into the requested file type.
    - **Error Cases:** Disk full or locked file; system alerts user to free space/close files.
10. **F10 - Natural Language AI Interface**
    - **Description:** A chat panel allowing users to ask specific questions about the codebase or a specific bug explanation.
    - **User Interaction:** User types queries into the AI interface panel.
    - **Input:** Natural language query and selected code context.
    - **Output:** Conversational AI response.
    - **AI Involvement:** AI-based.
    - **Workflow:** Chat text + code context sent to LLMClient, streamed back to the chat UI.

- **Error Cases:** Irrelevant prompts; agent restricts answers to programming context.

3. UML Class Diagram



4. Design Pattern Explanations

1. Singleton Pattern

Problem Addressed: Provides a unified global interface for managing LLM API connections, preventing redundant network traffic, safeguarding API keys, and enforcing rate limiting.

Participating Classes: LLMClient.

Class Roles: LLMClient limits instantiation to one internal reference (instance: LLMClient) and exposes global access using getInstance().

Pattern Justification: Centralized state management is necessary to control request rates and API credentials. Instantiating multiple clients would cause race conditions and wasteful resource allocation.

Trade-offs Without Pattern: Application components like summarization or vulnerability scanning would spin up independent API clients, making quota tracking, credential handling, and apiKey management unmaintainable.

2. Strategy Pattern

Problem Addressed: Enables diverse review operations (e.g., security audits vs. style checks) without polluting core classes with complex if/else branches.

Participating Classes: ReviewStrategy (Interface), SecurityStrategy, StyleStrategy, and ReviewAgent (Context).

Class Roles: ReviewStrategy specifies the executeStrategy() contract. SecurityStrategy and StyleStrategy encapsulate target analysis logic and prompts. ReviewAgent holds the active strategy object and executes it dynamically.

Pattern Justification: Promotes runtime flexibility by letting users switch dynamic review behaviors while fulfilling Open/Closed design principles.

Trade-offs Without Pattern: The core ReviewAgent becomes a monolithic class packed with conditional logic. Introducing new analysis types would require direct edits to the agent, risking code regressions.

3. Command Pattern

Problem Addressed: Encapsulates feedback interactions (accepting or dismissing findings) into standalone request objects for logging, execution, or undo operations.

Participating Classes: ReviewCommand (Interface), AcceptCommand, RejectCommand, and CodeReviewGUI (Invoker).

Class Roles: ReviewCommand defines the execute() signature. AcceptCommand and RejectCommand encapsulate the target ReviewItem alongside human feedback strings. CodeReviewGUI invokes these commands on user action.

Pattern Justification: Decouples UI action triggers (button clicks) from underlying business logic that modifies issue state and logs feedback.

Trade-offs Without Pattern: Frontend components would directly update ReviewItem states, causing tight coupling and complicating features like feedback tracking or action histories.

4. Observer Pattern

Problem Addressed: Delivers real-time interface updates across multiple views when underlying analysis results change, avoiding polling loops.

Participating Classes: Observer (Interface), CodeReviewGUI, CLIView, and ReviewReport (Subject).

Class Roles: ReviewReport holds core finding records. Observer mandates an updateUI() signature. CodeReviewGUI and CLIView register as observers to refresh automatically upon notifications.

Pattern Justification: Establishes an event-driven flow where the ReviewReport subject broadcasts state updates without coupling to specific view implementations.

Trade-offs Without Pattern: Views would either freeze the interface while waiting on synchronous background tasks or waste compute cycles continuously polling for updates.

5. Facade Pattern

Problem Addressed: Shields application controllers from complex internal operations like LLM client setup, payload orchestration, and strategy execution.

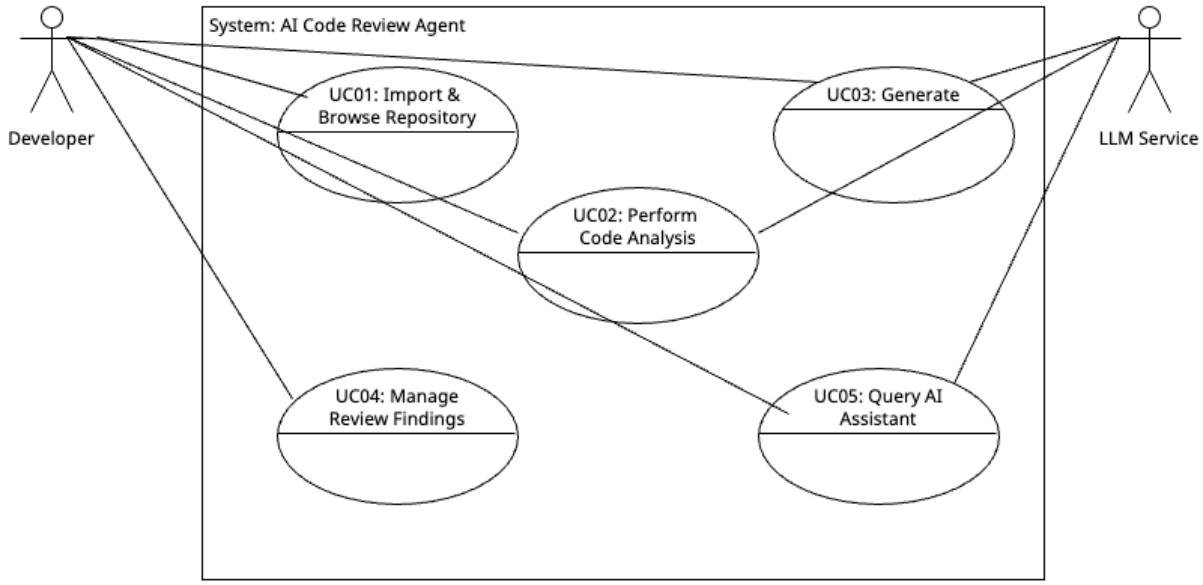
Participating Classes: ReviewAgent (Facade), LLMClient (Subsystem), ReviewStrategy (Subsystem), and AgentController (Client).

Class Roles: ReviewAgent exposes simplified endpoints like analyzeCode() or generateTests(), delegating execution details to LLMClient and ReviewStrategy. The AgentController relies solely on this facade interface.

Pattern Justification: Hides subsystem complexity behind a streamlined entry point, preserving overall system architecture readability.

Trade-offs Without Pattern: The AgentController would directly manage the LLMClient, construct prompt payloads, parse responses, and handle strategies, violating Single Responsibility and creating brittle code.

5. Use-Case Diagram



6. Detailed Use-Case Descriptions

Use Case ID: UC01

- **Use Case Name:** Import and Browse Repository
- **Actor(s):** Developer
- **Goal:** Ingest a local directory or remote repository to enable integrated file navigation.
- **Preconditions:** Application system is operational and target location credentials or network paths are available.
- **Trigger:** User submits the target repository locator and selects the action to import.
- **Main Success Scenario:**
    1. User inputs the file system location or remote Git address.
    2. System verifies path integrity and confirms directory permissions.
    3. System reads or checks out the target file repository structure.
    4. System renders the hierarchical directory tree within the user interface view.
- **Alternative/Exception Flows:** Invalid path string or permission denied; system prompts user to verify access controls.
- **Postconditions:** Source repository structural hierarchy is populated and primed for downstream operations.
- **Related Feature(s):** F01 (Repository Import & File Browsing).

Use Case ID: UC02

- **Use Case Name:** Perform Code Analysis
- **Actor(s):** Developer, LLM Service
- **Goal:** Analyze target source files to pinpoint logical bugs, evaluate formatting guidelines, and generate automated summaries.
- **Preconditions:** UC01 execution completed; project files are successfully mapped.
- **Trigger:** User highlights source content and triggers the manual review execution.
- **Main Success Scenario:**
    1. User initiates the code review request.
    2. System gathers active source text alongside configured project style policies.
    3. System constructs structured prompt payloads for execution by the AI service.
    4. LLM client processes payload and provides categorized defect records and textual summaries.
    5. System processes returned data models and populates findings within the active interface panel.
- **Alternative/Exception Flows:** API timeout; system retries operation automatically before issuing a connection warning alert.
- **Postconditions:** Structured summary notes and categorized code quality issues appear in the review panel.
- **Related Feature(s):** F02 (Code Summarization), F03 (Issue Detection), F06 (Coding-Standard Checking).

Use Case ID: UC03

- **Use Case Name:** Generate Code Modifications
- **Actor(s):** Developer, LLM Service
- **Goal:** Automatically produce functional unit tests or optimized multi-step refactoring proposals for designated methods.
- **Preconditions:** UC02 completed with flagged finding selected, or specific target function highlighted.
- **Trigger:** User selects "Suggest Refactor" for a detected issue or clicks "Generate Tests" on an isolated block.
- **Main Success Scenario:**
    1. User executes the generative code modification command.
    2. System constructs contextual inputs including signatures, function bodies, and issue flags.
    3. System delegates request generation directly to the API integration client.
    4. LLM client returns synthesized unit test suites or modified implementation snippets.
    5. System presents side-by-side comparison view displaying recommended updates.
- **Alternative/Exception Flows:** Unrecognized testing framework; system defaults to standard language libraries.
- **Postconditions:** Generated code variations or testing blocks become available for inspection and application.

- **Related Feature(s):** F04 (Refactoring Suggestions), F05 (Test Generation Suggestion).

Use Case ID: UC04

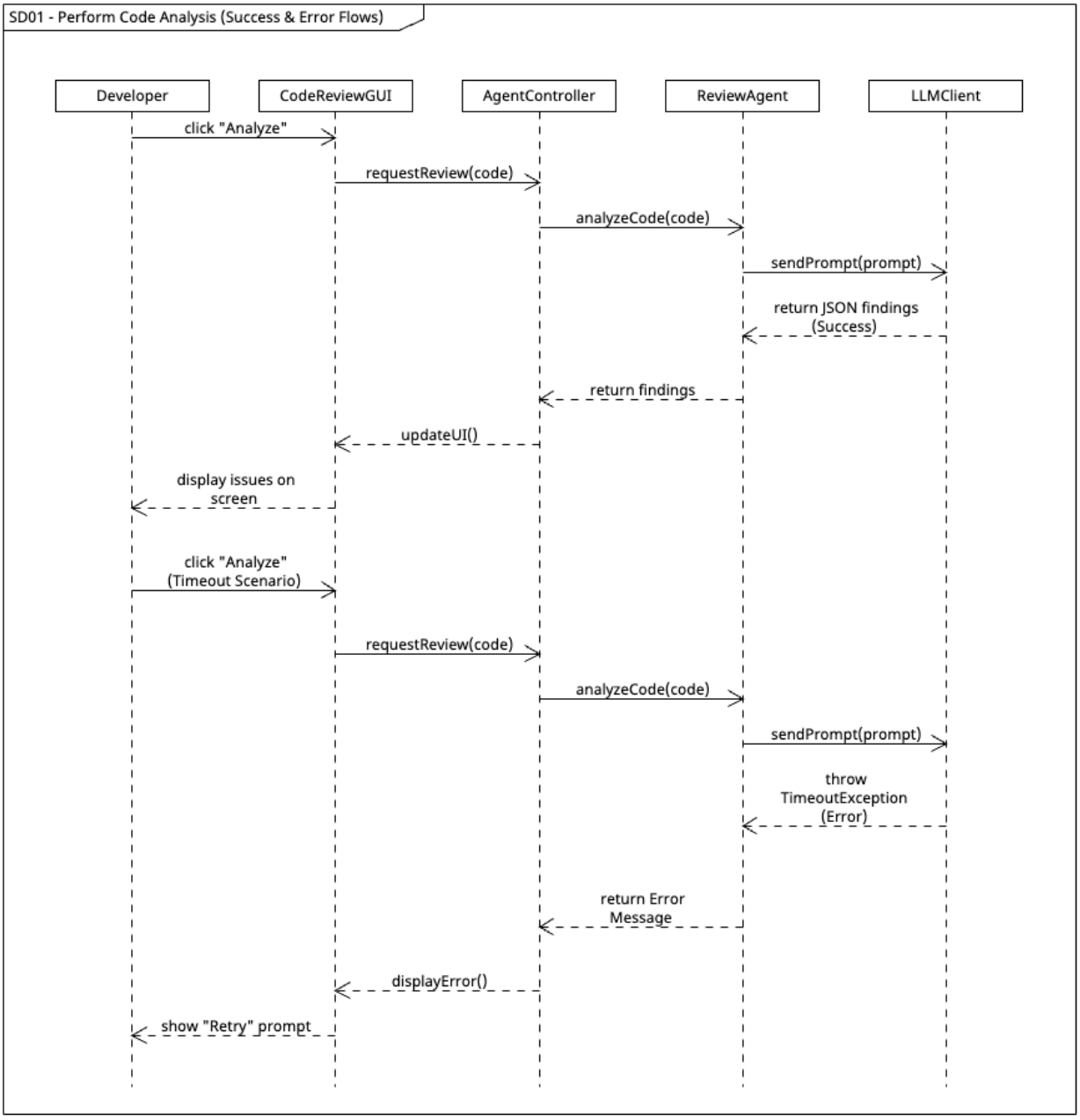
- **Use Case Name:** Manage Review Findings
- **Actor(s):** Developer
- **Goal:** Process user decisions on findings, evaluate version history shifts, and compile structured document reports.
- **Preconditions:** System contains active or persistent ReviewItem findings.
- **Trigger:** User selects accept/reject status, requests commit comparisons, or exports session results.
- **Main Success Scenario:**
    1. User executes specific finding management or export action.
    2. System dispatches corresponding command object to update state, compare diffs, or aggregate session state.
    3. System writes feedback logs to persistent storage or compiles export file.
    4. Interface updates automatically to clear resolved elements or deliver downloadable artifacts.
- **Alternative/Exception Flows:** Disk full or locked file; system alerts user to free space/close files.
- **Postconditions:** Finding state changes persist reliably, and exported report summaries save to disk.
- **Related Feature(s):** F07 (Review Acceptance/Rejection), F08 (Comparison Across Versions), F09 (Report Generation).

Use Case ID: UC05

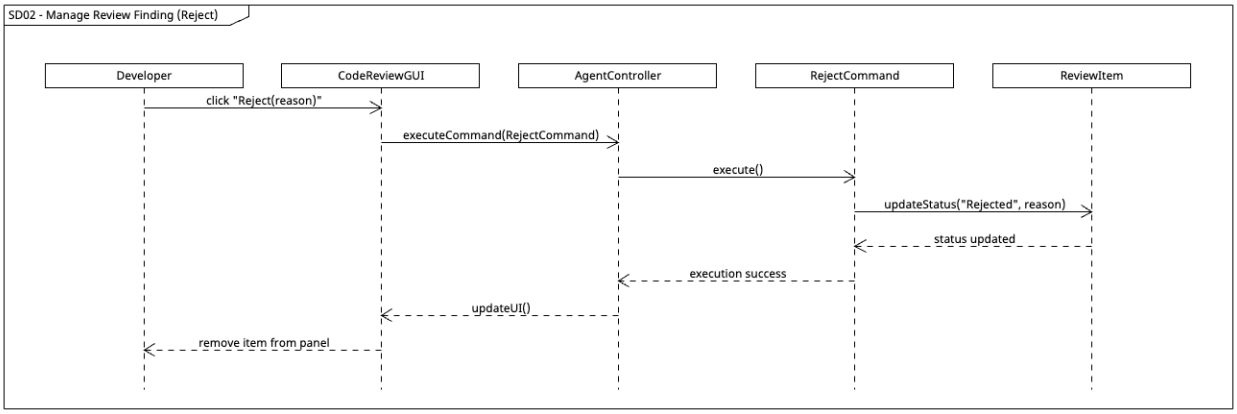
- **Use Case Name:** Query AI Assistant
- **Actor(s):** Developer, LLM Service
- **Goal:** Address natural-language questions related to implementation details or bug root causes.
- **Preconditions:** Source repository context is loaded within the active workspace.
- **Trigger:** User inputs query string into natural-language panel and submits message.
- **Main Success Scenario:**
    1. User enters query string into chat workspace.
    2. System attaches local code window state as contextual input.
    3. System sends request payload to the AI API client.
    4. LLM client synthesizes explanatory response.
    5. System streams conversational response back into chat view panel.
- **Alternative/Exception Flows:** Irrelevant prompts; agent restricts answers to programming context.
- **Postconditions:** Active dialogue panel updates with system response history.
- **Related Feature(s):** F10 (Natural Language AI Interface).

7. Sequence Diagrams

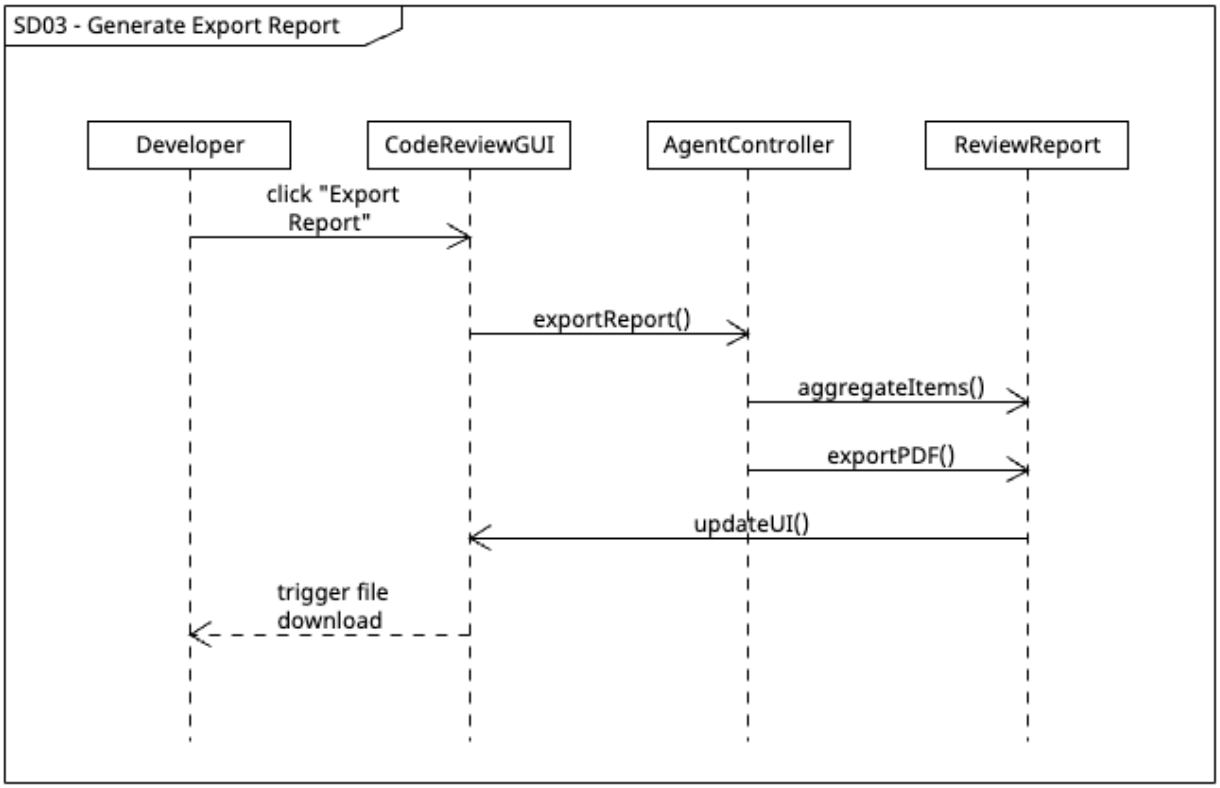
SD01 - Perform Code Analysis



SD02 - Manage Review Finding



SD03 - Generate & Export Report



8. Feature-to-Design Traceability Table

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Import & browse repository	Deterministic	UC01: Import & Browse	CodeReviewGUI, AgentController, RepoManager	handleImport(), loadFiles()	N/A	None
F02	Code summarization	AI	UC02: Perform Code Analysis	CodeReviewGUI, AgentController, ReviewAgent, LLMClient	requestReview(), analyzeCode(), sendPrompt()	SD01	Facade, Singleton
F03	Issue detection & categorization	AI	UC02: Perform Code Analysis	AgentController, ReviewAgent, SecurityStrategy, LLMClient, ReviewItem	analyzeCode(), executeStrategy(), sendPrompt()	SD01	Facade, Strategy, Singleton
F04	Refactoring suggestions	AI	UC03: Generate Code Modifications	CodeReviewGUI, AgentController, ReviewAgent, LLMClient	requestReview(), analyzeCode(), sendPrompt()	SD01	Facade, Singleton

F05	Test generation suggestion	AI	UC03: Generate Code Modifications	CodeReviewGUI, AgentController, ReviewAgent, LLMClient	requestReview(), generateTests(), sendPrompt()	SD01	Facade, Singleton
F06	Coding-standard check	Hybrid	UC02: Perform Code Analysis	AgentController, ReviewAgent, StyleStrategy, LLMClient	analyzeCode(), executeStrategy(), sendPrompt()	SD01	Strategy, Facade
F07	Review accept/reject & feedback	Deterministic	UC04: Manage Review Findings	CodeReviewGUI, AgentController, RejectCommand, AcceptCommand, ReviewItem	executeCommand(), execute(), updateStatus()	SD02	Command
F08	Comparison of reviews across versions	Hybrid	UC04: Manage Review Findings	AgentController, RepoManager, ReviewReport	getDiff(), aggregateItems()	SD03	None
F09	Review-history & report generation	Deterministic	UC04: Manage Review Findings	CodeReviewGUI, AgentController, ReviewReport, Observer	exportReport(), aggregateItems(), exportPDF(), updateUI()	SD03	Observer
F10	Natural language AI interface	AI	UC05: Query AI Assistant	CodeReviewGUI, AgentController, ReviewAgent, LLMClient	submitChat(), sendPrompt()	SD01	Facade, Singleton

9. Feature Implementation Explanations

F01 — Import & Browse Repository

- **Related Use Case:** UC01 — Import & Browse Repository
- **Related Sequence Diagram:** N/A (Simple UI to Controller call)
- **Classes involved:**
  - CodeReviewGUI — captures the user's input path or URL.
  - AgentController — coordinates the repository import request.
  - RepoManager — accesses the local file system or Git environment.
- **Important methods:**
  - CodeReviewGUI.handleImport()
  - AgentController.handleImport()
  - RepoManager.loadFiles()
- **Execution:**
  When the user enters a repository path in the GUI, the GUI triggers handleImport() on the AgentController. The controller delegates this operation to RepoManager.loadFiles(), which scans the directory and constructs a hierarchical list of files. This file tree structure is returned to the GUI to populate the repository browser UI.

F02 — Code Summarization

- **Related Use Case:** UC02 — Perform Code Analysis
- **Related Sequence Diagram:** SD01 — Perform Code Analysis
- **Classes involved:**
  - CodeReviewGUI — allows the user to highlight text and click "Summarize".
  - AgentController — routes the analysis request.
  - ReviewAgent — acts as the Facade to orchestrate the AI summarization.
  - LLMClient — manages the API communication with the model.
- **Important methods:**
  - CodeReviewGUI.requestReview()
  - AgentController.requestReview()
  - ReviewAgent.analyzeCode()
  - LLMClient.sendPrompt()

- **Execution:**
  The developer highlights a block of code and requests a summary. The GUI sends the code string to AgentController.requestReview(), which delegates it to ReviewAgent.analyzeCode(). The agent bundles the code into a targeted summarization prompt and sends it to the LLMClient. The string returned by the API is passed back up to the GUI and displayed in the review panel.

F03 — Issue Detection & Categorization

- **Related Use Case:** UC02 — Perform Code Analysis
- **Related Sequence Diagram:** SD01 — Perform Code Analysis
- **Classes involved:**
  - AgentController — processes the UI trigger.
  - ReviewAgent — dynamically manages the analysis process.
  - SecurityStrategy — defines the specific rules and prompts for bug/vulnerability detection.
  - LLMClient — performs the heavy AI reasoning.
  - ReviewItem — stores the structured data of a single detected bug.
- **Important methods:**
  - AgentController.requestReview()
  - ReviewAgent.analyzeCode()
  - SecurityStrategy.executeStrategy()
  - LLMClient.sendPrompt()
- **Execution:**
  When an analysis is triggered, AgentController asks ReviewAgent to scan the file. The agent applies the SecurityStrategy, formatting the code and querying the LLMClient for logic flaws and vulnerabilities. The LLM returns a JSON payload, which the agent parses into a list of ReviewItem objects categorized by severity (e.g., Critical, Warning). The GUI updates to display these items.

F04 — Refactoring Suggestions

- **Related Use Case:** UC03 — Generate Code Modifications
- **Related Sequence Diagram:** SD01 — Perform Code Analysis
- **Classes involved:**
  - CodeReviewGUI — displays the side-by-side code diff.
  - AgentController — coordinates the refactoring workflow.
  - ReviewAgent — constructs the refactoring prompt.
  - LLMClient — generates the new code snippet.
- **Important methods:**
  - AgentController.requestReview()
  - ReviewAgent.analyzeCode()
  - LLMClient.sendPrompt()
- **Execution:**
  The user clicks "Suggest Refactor" on a specific ReviewItem. The request flows through

the AgentController to the ReviewAgent, which asks the LLMClient for optimized code replacements based on the detected issue. The LLM returns an executable code block, which the GUI renders in a side-by-side comparison view against the original code.

F05 — Test Generation Suggestion

- **Related Use Case:** UC03 — Generate Code Modifications
- **Related Sequence Diagram:** SD01 — Perform Code Analysis
- **Classes involved:**
  - CodeReviewGUI — collects the user's function selection.
  - AgentController — directs the test generation request.
  - ReviewAgent — manages the test generation rules.
  - LLMClient — writes the unit tests.
- **Important methods:**
  - AgentController.requestReview()
  - ReviewAgent.generateTests()
  - LLMClient.sendPrompt()
- **Execution:**
  The developer highlights a function signature and clicks "Generate Tests". AgentController invokes ReviewAgent.generateTests(). The agent crafts a specific prompt outlining the function and edge cases, sending it to LLMClient. The API returns a block of unit test code, which is routed back to the UI for the developer to copy or save.

F06 — Coding-Standard Check

- **Related Use Case:** UC02 — Perform Code Analysis
- **Related Sequence Diagram:** SD01 — Perform Code Analysis
- **Classes involved:**
  - AgentController — initiates the standard check.
  - ReviewAgent — orchestrates the strategy swap.
  - StyleStrategy — defines formatting rules and naming conventions.
  - LLMClient — evaluates code style.
  - ReviewItem — stores style violations.
- **Important methods:**
  - AgentController.requestReview()
  - ReviewAgent.analyzeCode()
  - StyleStrategy.executeStrategy()
  - LLMClient.sendPrompt()
- **Execution:**
  Similar to issue detection, but the ReviewAgent dynamically loads the StyleStrategy. The agent combines the raw code with project-specific .rules and asks the LLMClient to identify formatting or naming violations. The parsed results are instantiated as ReviewItem objects tagged as "Style Warnings" and displayed.

F07 — Review Acceptance/Rejection & Human Feedback Capture

- **Related Use Case:** UC04 — Manage Review Findings
- **Related Sequence Diagram:** SD02 — Manage Review Finding (Reject)
- **Classes involved:**
  - CodeReviewGUI — captures the button click and feedback text.
  - AgentController — routes the command execution.
  - RejectCommand / AcceptCommand — encapsulates the user action.
  - ReviewItem — the core object being modified.
- **Important methods:**
  - AgentController.executeCommand()
  - RejectCommand.execute()
  - ReviewItem.updateStatus()
- **Execution:**
  The developer clicks "Reject" on an issue and provides a text reason. The GUI creates a RejectCommand object and passes it to AgentController.executeCommand(). The command calls its execute() method, which invokes ReviewItem.updateStatus("Rejected", reason). The GUI then clears the resolved item from the active view.

F08 — Comparison of Reviews Across Versions

- **Related Use Case:** UC04 — Manage Review Findings
- **Related Sequence Diagram:** SD03 — Generate Export Report (Variant)
- **Classes involved:**
  - AgentController — coordinates the comparison process.
  - RepoManager — handles deterministic version differences.
  - ReviewReport — holds the aggregated finding data.
- **Important methods:**
  - AgentController.requestReview()
  - RepoManager.getDiff()
  - ReviewReport.aggregateItems()
- **Execution:**
  The user selects two commits in the UI to compare. AgentController calls RepoManager.getDiff() to extract the changed files deterministically. It then compiles the active ReviewItems associated with those changed files into a unified ReviewReport, highlighting the delta in unresolved issues between the two versions.

F09 — Review-History & Report Generation

- **Related Use Case:** UC04 — Manage Review Findings
- **Related Sequence Diagram:** SD03 — Generate Export Report
- **Classes involved:**
  - CodeReviewGUI — the observer waiting for the report to finish.
  - AgentController — coordinates the export action.
  - ReviewReport — aggregates data and writes the PDF.
  - Observer — the interface allowing UI updates.

- **Important methods:**
  - CodeReviewGUI.exportReport()
  - AgentController.exportReport()
  - ReviewReport.aggregateItems()
  - ReviewReport.exportPDF()
  - Observer.updateUI()
- **Execution:**
  The user clicks "Export Report". AgentController instructs ReviewReport to run aggregateItems() to collect all findings from the current session. The report then executes exportPDF() to format and save the file. Finally, it notifies its Observers (the GUI) via updateUI(), which alerts the user that the download is complete.

F10 — Natural Language AI Interface

- **Related Use Case:** UC05 — Query AI Assistant
- **Related Sequence Diagram:** SD01 — Perform Code Analysis (Variant for Chat)
- **Classes involved:**
  - CodeReviewGUI — captures the chat query.
  - AgentController — manages the chat routing.
  - ReviewAgent — provides code context to the AI.
  - LLMClient — processes the natural language.
- **Important methods:**
  - CodeReviewGUI.submitChat()
  - AgentController.requestReview()
  - LLMClient.sendPrompt()
- **Execution:**
  The developer types a specific question into the chat panel. submitChat() sends the query along with the currently active file context to the AgentController. The ReviewAgent bundles this into a conversational prompt, and LLMClient fetches the natural language explanation, streaming the answer directly back into the GUI chat window.